Generate Random Number

Section 4.10

Method to Generate a Random Int

int rand (void);

- Returns a pseudo-random integral number in the range between 0 and RAND_MAX. RAND_MAX is defined in <cstdlib>.
- code to print 3 random ints

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main()
{
    //print out 3 random ints
    for (int i = 0; i < 3; i++)
        cout << rand() << endl;

    return 0;
}
```

Method srand

void srand (unsigned int seed);

- **Initialize random number generator**

A sequence of random integers are generated from that seed.

- In order to generate random-like numbers, srand is usually initialized to some distinctive runtime value, like the value returned by function time (declared in header <ctime>).

```
/* initialize random seed: */
srand (time(NULL)); //time(NULL) gets the current time
```

Use srand and rand together: code link

- To see different random integers in different running time, call srand(time(NULL)); once before calling rand().

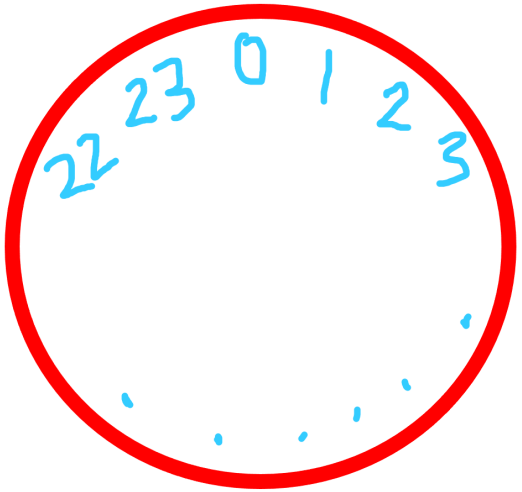
```
#include <iostream>
#include <cstdlib> //srand, rand, RAND_MAX
#include <ctime> //time
using namespace std;

int main()
{
    srand(time(NULL)); //same as srand(time(0));
    //print out 3 random ints
    for (int i = 0; i < 3; i++)
        cout << rand() << endl;

    return 0;
}
```

Throw a dice = generate random int in [1, 6]

- Hint: time is infinite, but how to label time by hours as military time?
  - Military time uses the 24-hour clock, which numbers the hours of the day from 00–23. For example, 6 AM is 0600 hours, and 1 PM is 1300 hours.
- How many integers in [1, 6]?



Generate a random int in [1, 6]: II

- What is the result of rand() % 6?
- Given a random int in [0, 5], how to get a random int in [1, 6]?

- Image source: https://battattoys.com/

Generate a random integer in [start, end]

- How many integers in [start, end]?
- How to generate a random integer in [0, end – start]?
- How to generate a random integer in [start, end]?

Generate a random floating point number in [start, end]

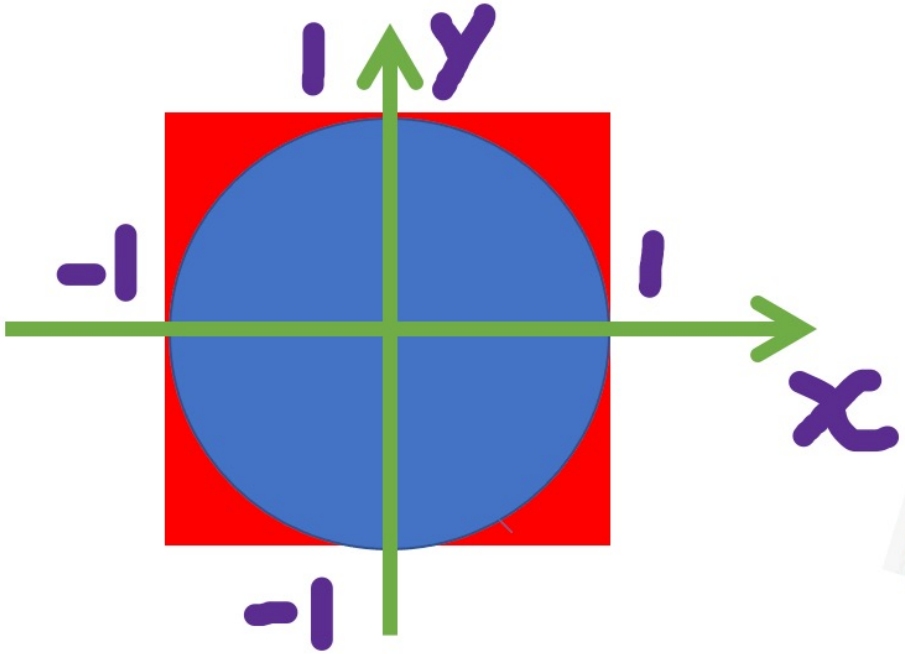
- Generate a random floating point number in [0, 1].

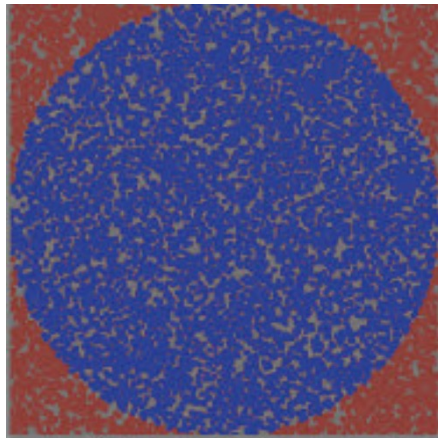
- Generate a random floating point number in [0, end – start]. It is like to draw a line in map by scaling up (or scale down if end-start <1)

- Generate a random floating point number in [start, end].

Use Monte Carlo simulation to estimate PI

- Given a unit circle centered at a square.

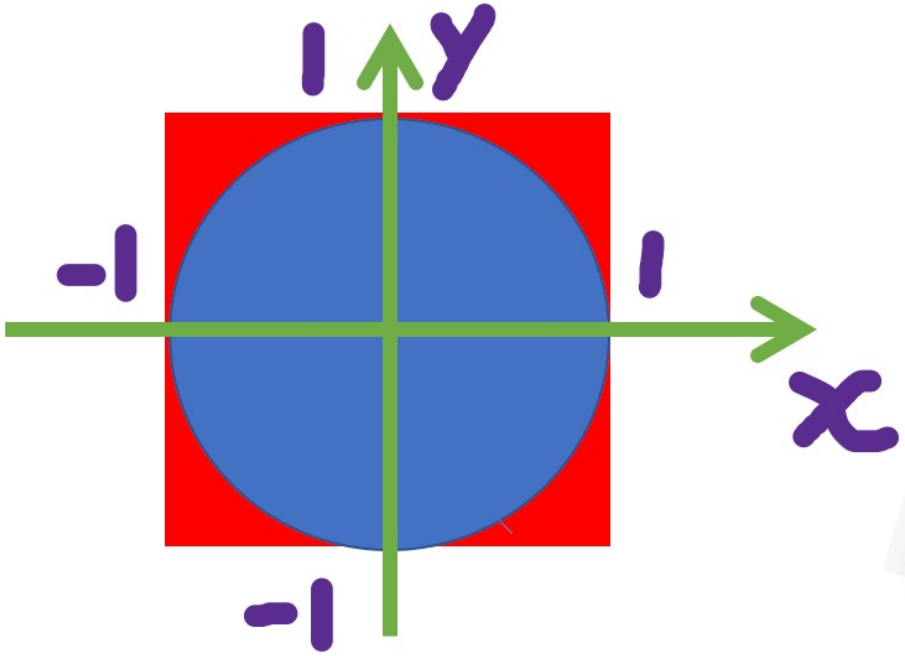


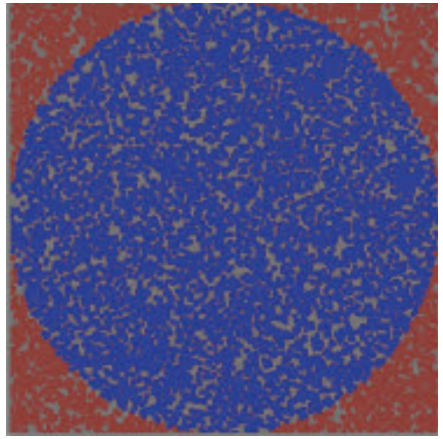


- What is the area of circle? What is the area of the square?

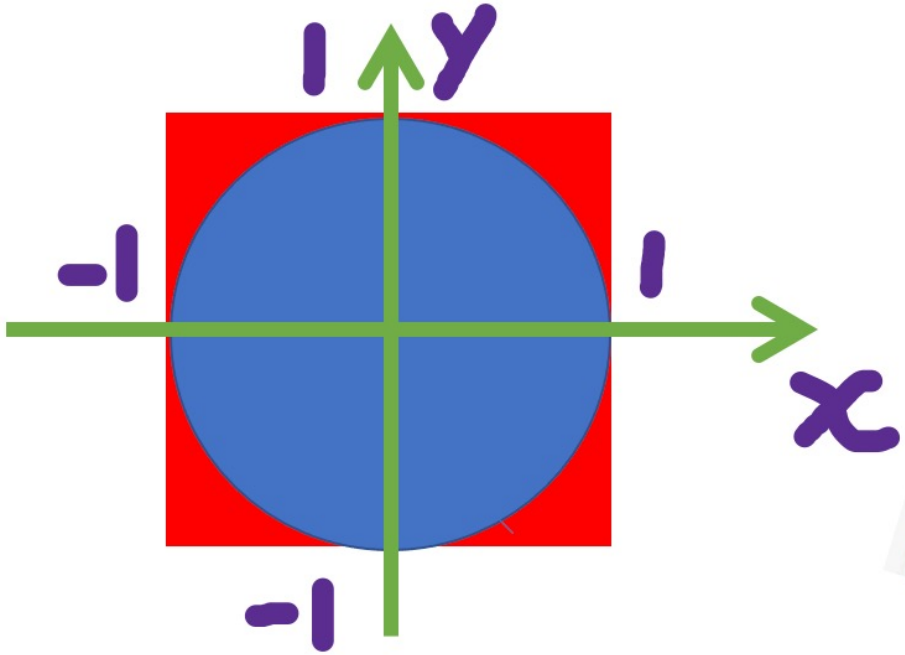
Use Monte Carlo simulation to estimate PI: II

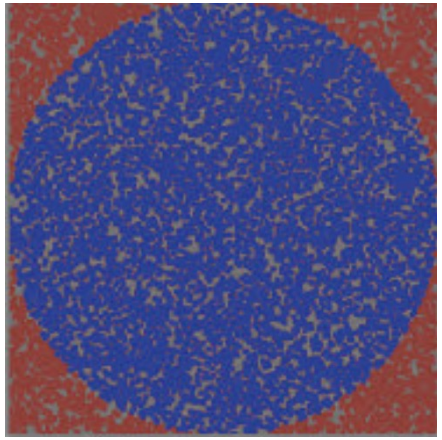
- Throw a dart many times (say, 10000 or more) at the following picture, where x and y are independent random floating point number in [-1, 1].





Use Monte Carlo simulation to estimate PI: III





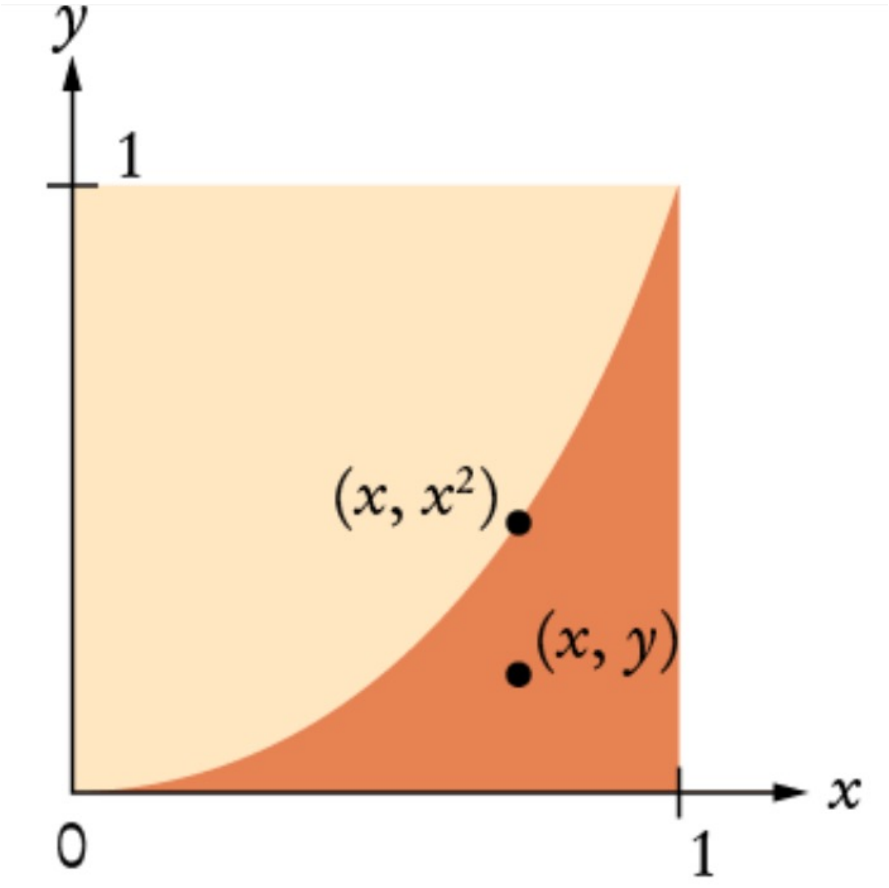
- In the right-hand picture, a dot is like the mark left by the dart.
- What is the chance (aka probability) that the dart falls in the circle?
- How to estimate the probability?

Pseudo-code of Monte Carlo Simulation of PI

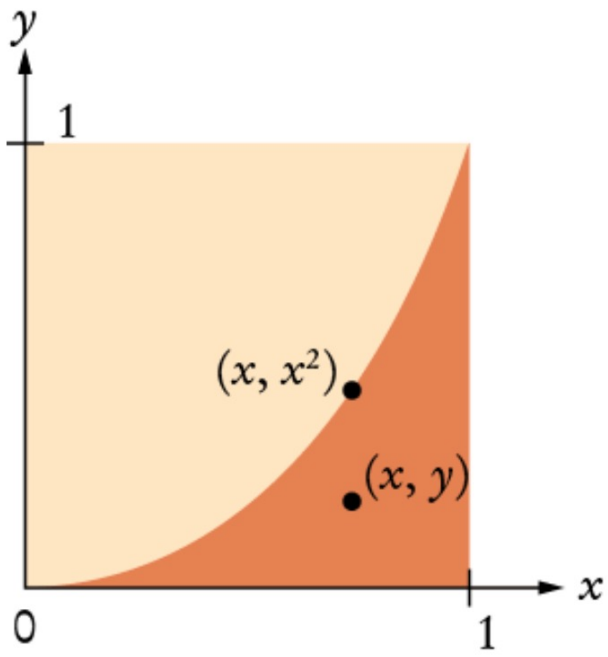
```
Set numThrows to be some sufficient big int, say, 10000.
Let numHits to be the number of times that the dart hits the circle, initialize
it to be zero.
For each throw
Begin
        Generate a random floating point number x in [-1, 1].
        Generate a random floating point number y in [-1, 1].
        if (x, y) falls inside or on the edge of the circle
                increase numHits by 1
End
Estimate PI by numHits and numThrows.
```

Another example

- Estimate the area under $y = x^2$ when x in [0, 1] and y in [0, 1].



Correction: p136 of textbook

- Courtesy Soren Soroush
- 


```
const int TRIES = 10000;
int hits = 0;
for (int i = 1; i <= TRIES; i++)
{
   // Generate two random numbers between 0 and 1
  double x = rand() * 1.0 / RAND_MAX;
  double y = rand() * 1.0 / RAND_MAX;
  if (...) { hits++; }
}
double area_estimate = ...;
```

What should be used for the two ... expressions?

- ☐ `x * x < y` and `4.0 * hits / TRIES`
- ☐ `x * x > y` and `4.0 * hits / TRIES`
- ☐ `x * x < y` and `hits / TRIES`
- ☐ `x * x > y` and `hits / TRIES`

Easier and better random generator in C++11

- **Random** (see https://www.cplusplus.com/reference/random/)

This header introduces random number generation facilities.
This library allows to produce random numbers using combinations of *generators* and *distributions*:

- **Generators:** Objects that generate uniformly distributed numbers.
- **Distributions:** Objects that transform sequences of numbers generated by a generator into sequences of numbers that follow a specific random variable distribution, such as uniform, Normal or Binomial.

```
std::default_random_engine generator;
std::uniform_int_distribution<int> distribution(1,6);
int dice_roll = distribution(generator);  // generates number in the range 1..6
```